Language Models for Information Retrieval

Topic 2 · Query likelihood and smoothing

Group discussion · Information Web Retrieval and Analytics

2026-11-03

The search problem

A user types **“language model probability.”**

Our collection contains documents about unigrams, query likelihood, bigrams, and perplexity.

How can we rank the documents using the probability of the query?

The query likelihood idea

1. Estimate a language model for each document.
2. Calculate how likely that model is to generate the query.
3. Rank documents by that likelihood.

Input: a query and a document collection.
Output: documents ordered by score.

Connection to Lecture 2

Lecture 2	Our retrieval model
Bag-of-words	Count terms in each document
N-gram probability	Estimate the probability of query text
Log probabilities	Add log scores for numerical stability
Unseen events and smoothing	Give partial matches a usable score

Our choice: a unigram model, so word order is ignored.

Course lecture: slides 4–8 and 12–20.

Earlier retrieval methods

Method	How it matches	Main gap
Boolean AND	Every required term must appear	Partial matches are rejected
TF-IDF cosine	Similarity of weighted term vectors	Depends on exact vocabulary
Query likelihood	Probability under each document model	Needs reliable probability estimates

Ponte and Croft introduced a language modeling approach to IR in **1998**.

Scoring a document

$$P_{\text{MLE}}(w \mid d) = \frac{c(w, d)}{|d|}$$

$$s(d, q) = \sum_{w \in q} c(w, q) \log P(w \mid d)$$

- $c(w, d)$ counts a word in the document. $|d|$ is its token count.
- Sum over distinct query terms. Repeated terms count again.
- **Higher is better:** a log score of −4 ranks above −8.

A missing word makes the whole product zero

Query: **“search documents”**

Document: **“language models rank documents”**

$$P_{\text{MLE}}(\text{search} \mid d) = 0$$

$$P(q \mid d) = 0 \times P(\text{documents} \mid d) = 0$$

Many partial matches tie at zero. In log space, their score is $-\infty$.

Collection-based smoothing

$$P_\mu(w \mid d) = \frac{c(w,d) + \mu P(w \mid C)}{|d| + \mu}$$

Dirichlet: μ controls collection-based pseudo-counts.

$$P_\lambda(w \mid d) = (1-\lambda)\frac{c(w,d)}{|d|} + \lambda P(w \mid C)$$

Jelinek–Mercer: λ is a fixed **background weight** here.

$P(w \mid C) = c(w,C)/|C|$. Collection frequencies replace the uniform background used by add-one smoothing.

A worked calculation

Collection: **14 tokens**, with `search` twice and `documents` three times.

Document C: **“language models rank documents”**, length 4. Set $\mu = 5$.

$$P(\text{search} \mid C_{doc}) = \frac{0 + 5(2/14)}{4 + 5} \approx 0.0794$$

$$P(\text{documents} \mid C_{doc}) = \frac{1 + 5(3/14)}{4 + 5} \approx 0.2302$$

$$P(q \mid C_{doc}) \approx 0.0183, \qquad s \approx -4.003$$

Parameters and assumptions

- We use $\mu = 20$ and $\lambda = 0.2$ in the main experiment.
- Larger μ gives more weight to collection frequencies.
- Unigrams ignore order and do not identify synonyms.
- We report and drop terms absent from the collection.

All query terms unknown: return no ranking.

Experiment setup

15 documents · 5 queries · declared binary relevance labels

Choice	What we do
Preprocessing	Lowercase alphabetic words, no stemming
Baseline	TF-IDF cosine similarity
Language models	MLE, JM (0.2), Dirichlet (20)
Evaluation	P@3, Recall@3, MAP, MRR

This is a teaching collection with our own labels, not a benchmark.

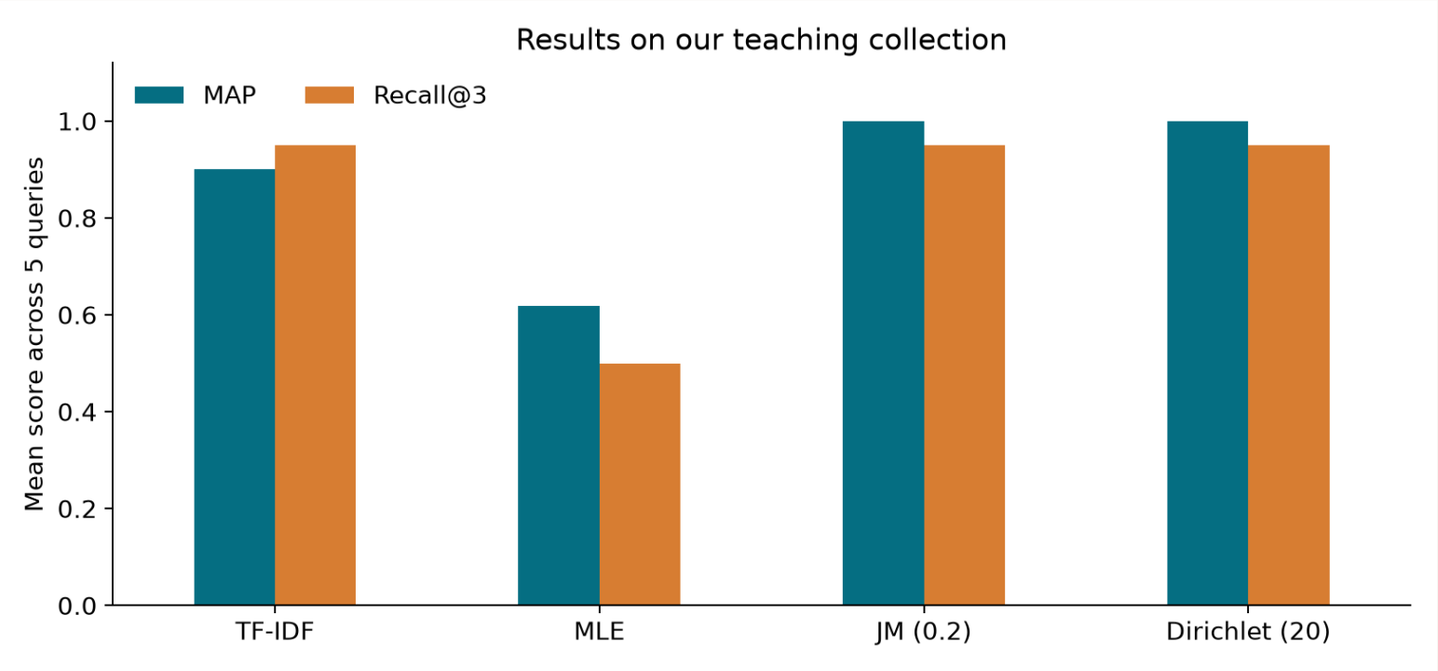
Python experiment

```
experiment = RetrievalExperiment(documents)

for query in queries:
    ranking, unknown = experiment.rank(
        query["text"], method="dirichlet", mu=20
    )
    metrics = evaluate(ranking, query["relevant"])
    print(query["id"], ranking[:3], metrics)
```

The notebook includes the full implementation, outputs, and checks.

Results on our collection



Method	MAP	Recall@3
TF-IDF	0.900	0.950
MLE	0.619	0.500
JM	1.000	0.950
Dirichlet	1.000	0.950

Smoothed models improve the first relevant rank on **one of five queries**.

Why Q5 changes rank

Query: **“car coverage”** · Intended need: vehicle insurance

Method	First result	Second result
TF-IDF	D14: car parking	D13: automobile insurance
Dirichlet	D13: automobile insurance	D14: car parking

D14 repeats `car`. D13 contains `coverage`, but uses `automobile`.

What we learned: smoothing preserves useful partial evidence. It did not learn that `car` and `automobile` are synonyms.

Sensitivity to smoothing

μ	MAP	Recall@3
0.1	1.000	0.950
1	1.000	0.950
5	1.000	0.950
20	1.000	0.950
100	1.000	0.950
1000	1.000	0.950

We vary μ on the same five queries. This is **sensitivity analysis**, not held-out tuning.

For a real study, choose parameters on development queries and report results on separate test queries.

Limitations and comparison

Limitation	Consequence or next step
Exact vocabulary	`car protection` still favors parking
Word order ignored	Unigrams cannot distinguish reordered words
Parameter choice	Tune smoothing on development queries
Tiny, manually judged collection	Use an independent judged benchmark

TF-IDF is a simple baseline. BM25 adds term-frequency saturation and length normalization, but also relies mainly on lexical matching.

A documented IR system: Indri

The **Lemur Project's Indri** uses language modeling for retrieval.

Its documentation describes query-likelihood scoring with **Dirichlet smoothing** and returned log scores.

Its documented default is $\mu = 2500$. That setting is not automatically suitable for our short teaching documents.

Lemur Project: Indri Document Scoring

Conclusion and next steps

Query likelihood ranks documents by how likely their word distributions are to generate the query.

Smoothing makes partial matches usable. Our experiment shows a benefit on one query and a remaining vocabulary failure.

Next step: evaluate on a judged benchmark, tune on development queries, and compare BM25 and query expansion.

References

- DORN Dawin (2026). *Lecture 2: NLP Foundations I: Bag-of-Words and Language Models*. ITC, slides 4–8, 12–20.
- *Group Discussion Topics: Retrieval Models*. Supplied handout, pp. 1 and 3.
- Manning, Raghavan & Schütze (2008). *Introduction to Information Retrieval*, Chapter 12.
- Ponte & Croft (1998). *A Language Modeling Approach to Information Retrieval*. SIGIR paper.
- Zhai & Lafferty (2001). *A Study of Smoothing Methods for Language Models Applied to Ad Hoc Information Retrieval*. SIGIR paper.
- Lemur Project. Indri scoring and query language.
- Scikit-learn. TfidfVectorizer API.